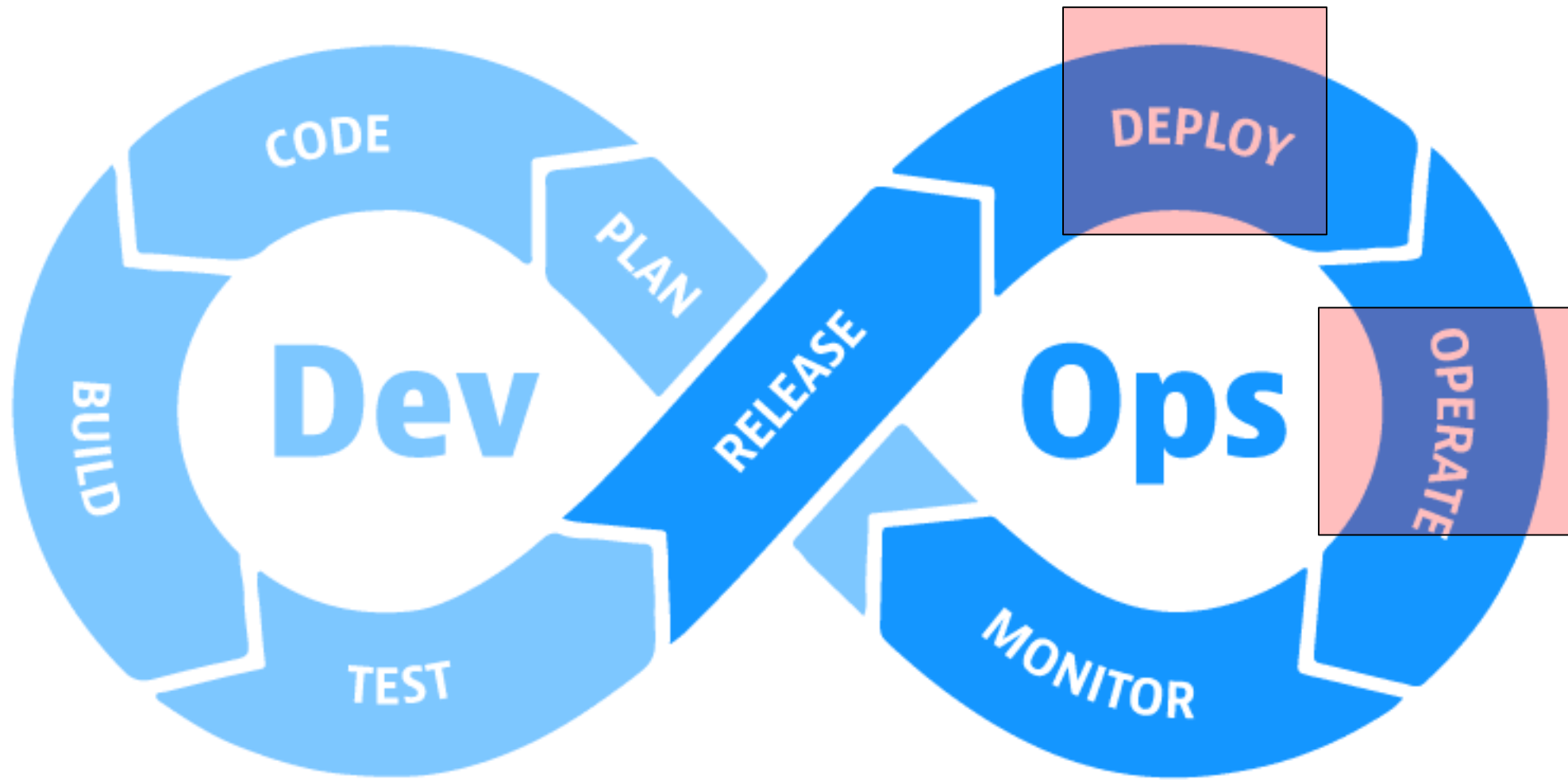


Developer Operations (devops) Platform



Build and Release

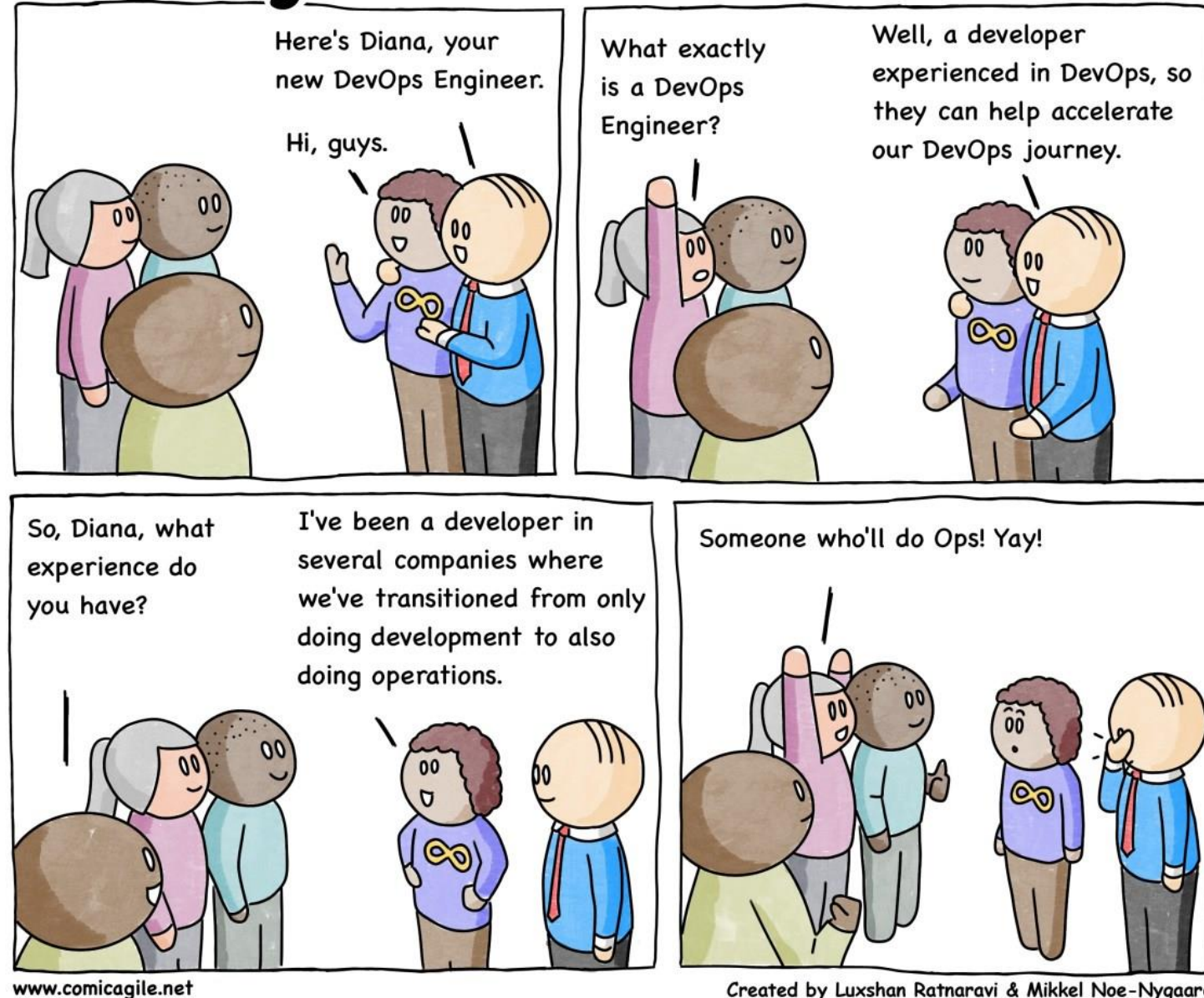


Where do you deploy your software?

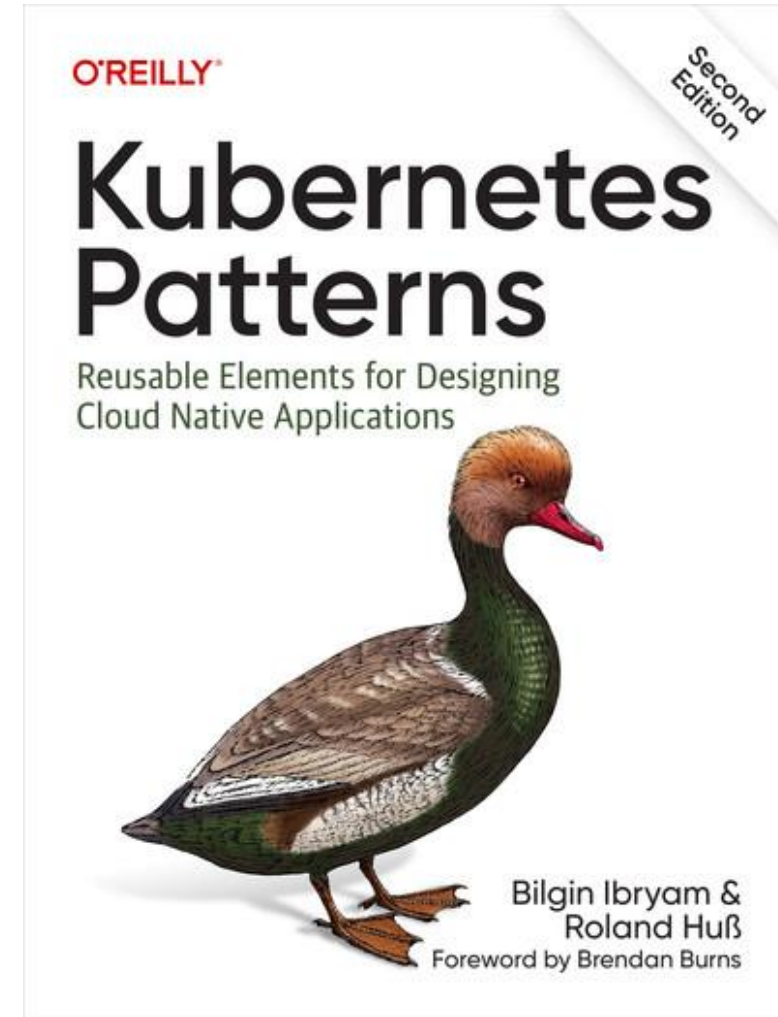
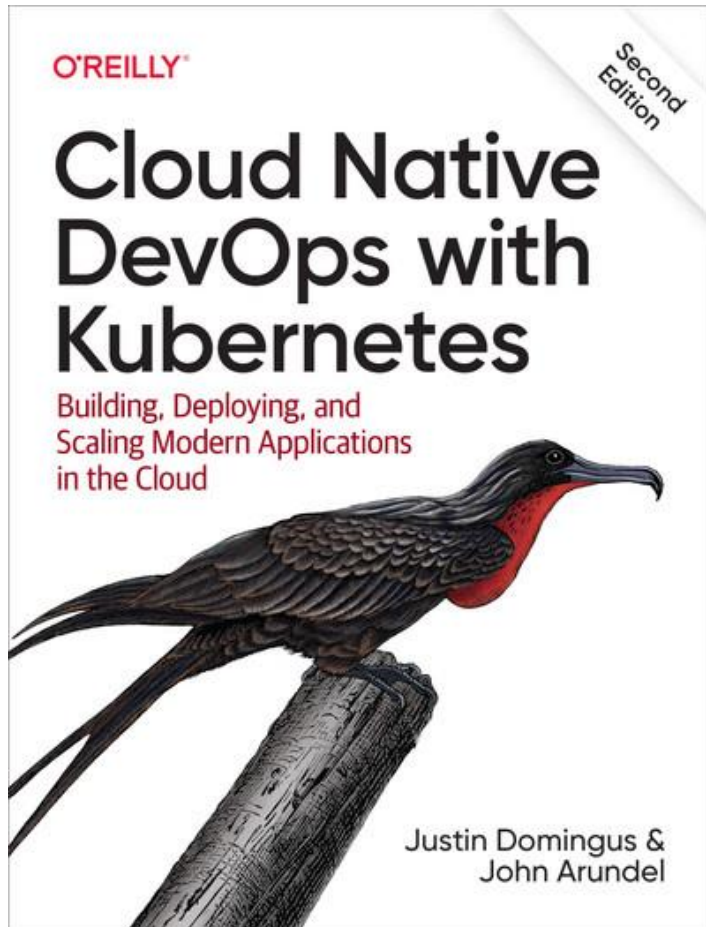
How do you run your software?

How do you ensure, the platform is stable?

Comic Agilé



Content for this lecture



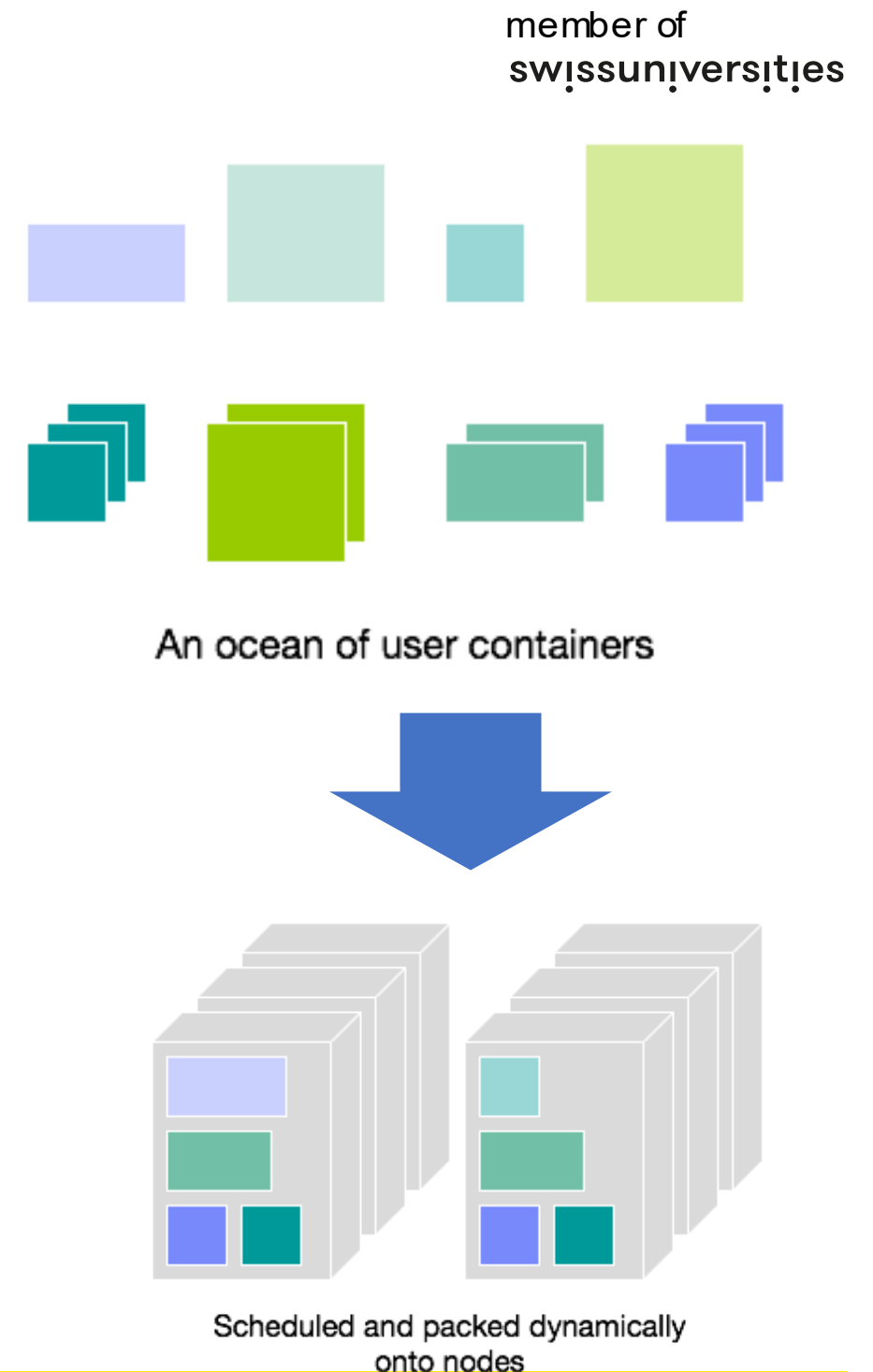
<https://learning.oreilly.com/library/view/kubernetes-patterns-2nd/9781098131678/>

<https://learning.oreilly.com/library/view/cloud-native-devops/9781098116811/>

<https://github.com/cloudnativdevops/demo>

Container Orchestration

- **Start and Stop Container**
- **Monitor** Container and restart if needed
- Publish new versions of a container (deploy)
- Operate multiple version of the same container (Zero-Downtime, Rolling-Upgrade)
- Enforce Security
- Provide **Persistent Storage**
- Provide **Software-Defined Networking**
- **Communication**, internally and externally
- Interaction with Orchestration



What is Kubernetes?

"Kubernetes is an open-source system for automating deployment, scaling, and management of containerized applications."

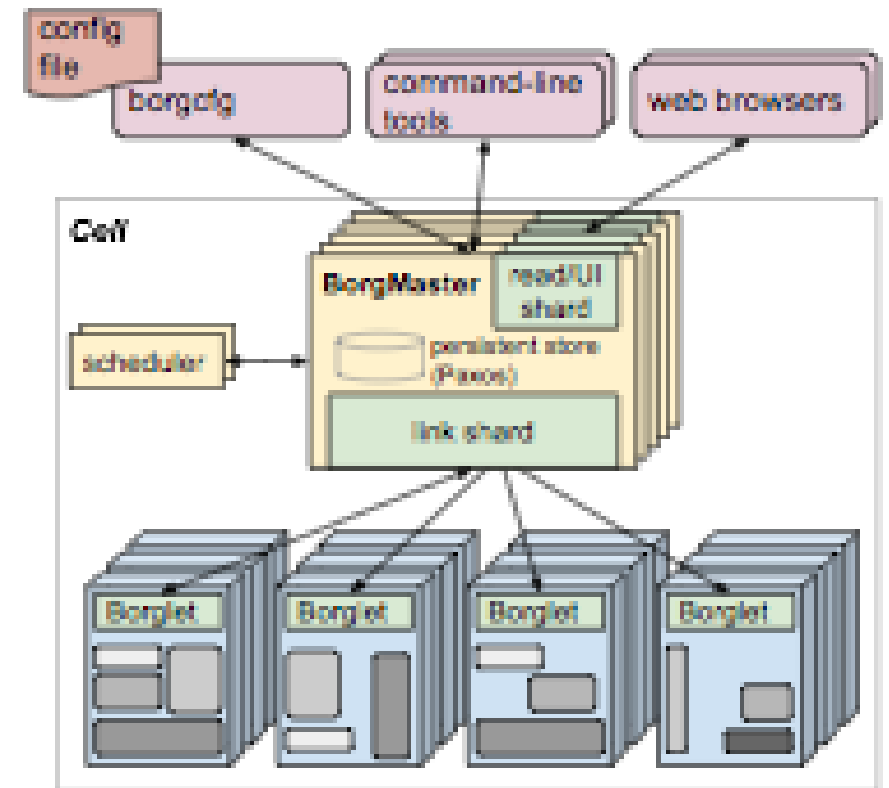


- Kubernetes (also referred to as **k8s**) originates from Greek, meaning helmsman or pilot, we can think of Kubernetes as the pilot on a ship of containers.
- Kubernetes is highly inspired by the **Google Borg system**, a container and workload orchestrator for its global operations, Google has been using for more than a decade (since 2003/2004).
- With its v1.0 release in July **2015**, Google donated Kubernetes to the *Cloud Native Computing Foundation (CNCF)*
- Kubernetes is written in the Go language and licensed under the Apache License Version 2.0.

Origin

- In 2015 Google publishes a paper on their Borg cluster manager
- Used internally since around 2005
- Borgmaster
 - Manages 10'000 machines
 - Needs to place new tasks at a rate of 10'000 tasks per minute
 - Uses 10-14 CPU cores and up to 50 GB RAM

<https://research.google/pubs/pub43438/>



>10'000 machines
> 100'000 jobs
> 1'000 types of applications

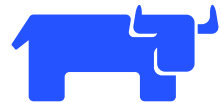
What is Kubernetes?

Nowadays, there is a wide variety of Kubernetes distributions available, including both on-premise software solutions and cloud hosted services.



K3S

MicroK8s



RANCHER



VMware Tanzu



Red Hat
OpenShift
Dedicated



Google Kubernetes Engine



Red Hat
OpenShift



Amazon EKS



Azure Kubernetes Service (AKS)

Usecase in SBB

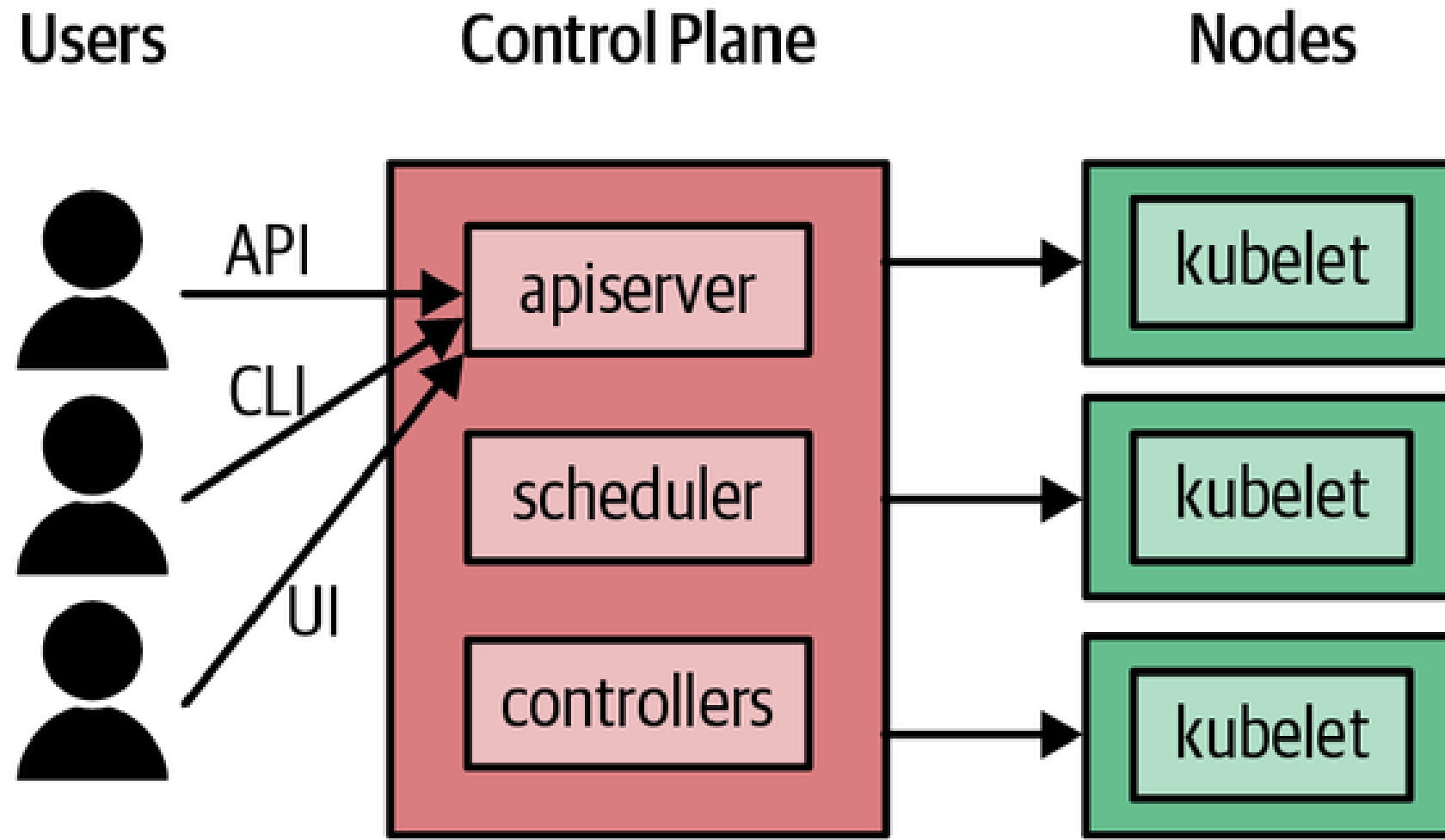
- 42 clusters, 800 nodes, 11'000 vCPUs, tons of memory
- 6 Development clusters
- 19 Test clusters
- 17 Production clusters
- Multi cloud strategy
 - AWS
 - Azure
 - Open Telekom Cloud
- Applications: mostly Spring Boot, Kafka
- PostgreSQL, Vault
- Purely Openshift...

Peoplecount for Platform: 7

<https://github.com/SchweizerischeBundesbahnen/container-platform-base>

<https://www.youtube.com/watch?v=eT5Dz3fziQY&t>

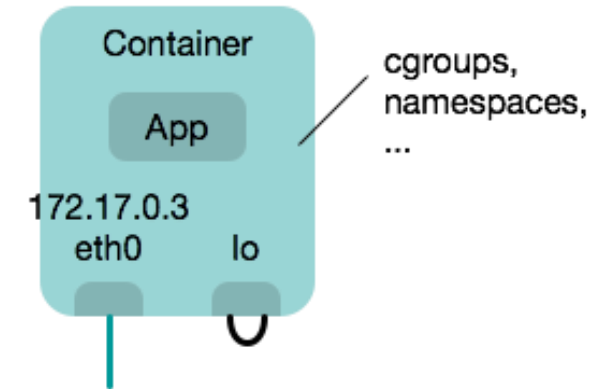
Kubernetes Architecture



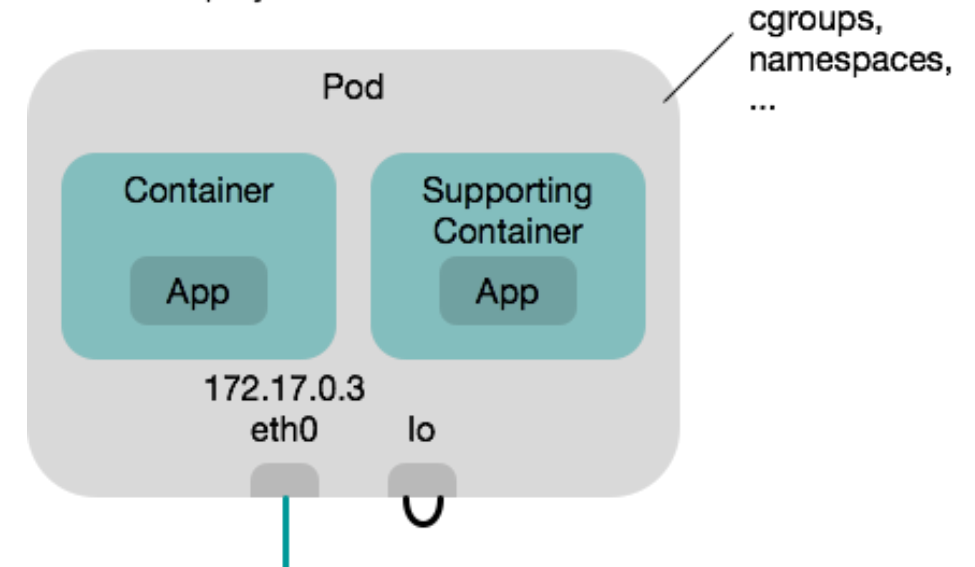
Pods

- The atomic unit of deployment in Kubernetes is the **Pod**.
- A Pod contains **one or more containers**.
- If a Pod has multiple containers
 - Kubernetes guarantees that they are scheduled on the **same** cluster node.
 - The containers **share** the same Pod environment
 - IPC namespace, shared memory, storage volumes, network stack, etc.
 - IP address
 - If containers need to talk to each other **within** the Pod, they can simply use the *localhost* interface.

Unit of deployment in Docker: Container

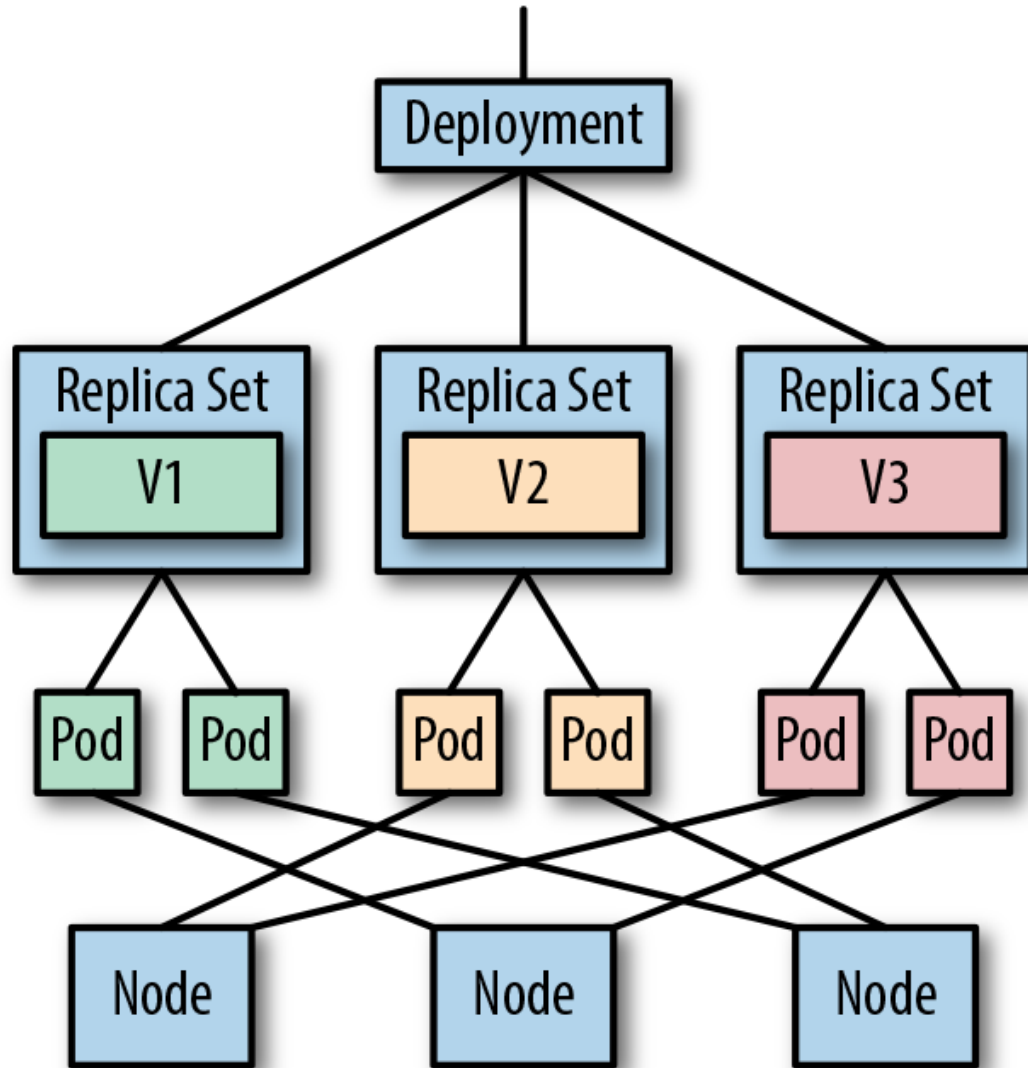


Unit of deployment in Kubernetes: Pod



Resources

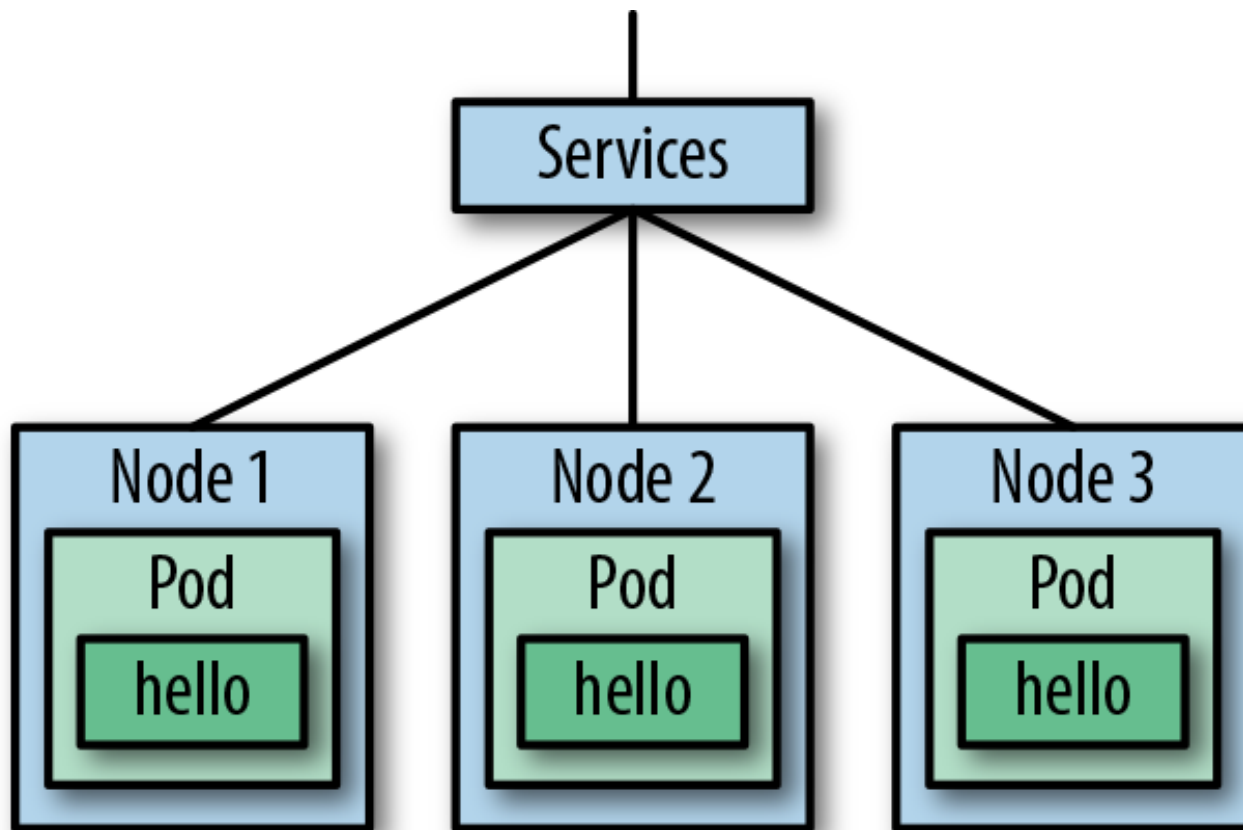
Pod and Deployment



```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: demo
spec:
  replicas: 3
  selector:
    matchLabels:
      app: demo
  template:
    metadata:
      labels:
        app: demo
    spec:
      containers:
        - name: demo
          image: cloudfnative/demo:hello-pod
          ports:
            - containerPort: 8888
  
```

How to access Container? Service



```

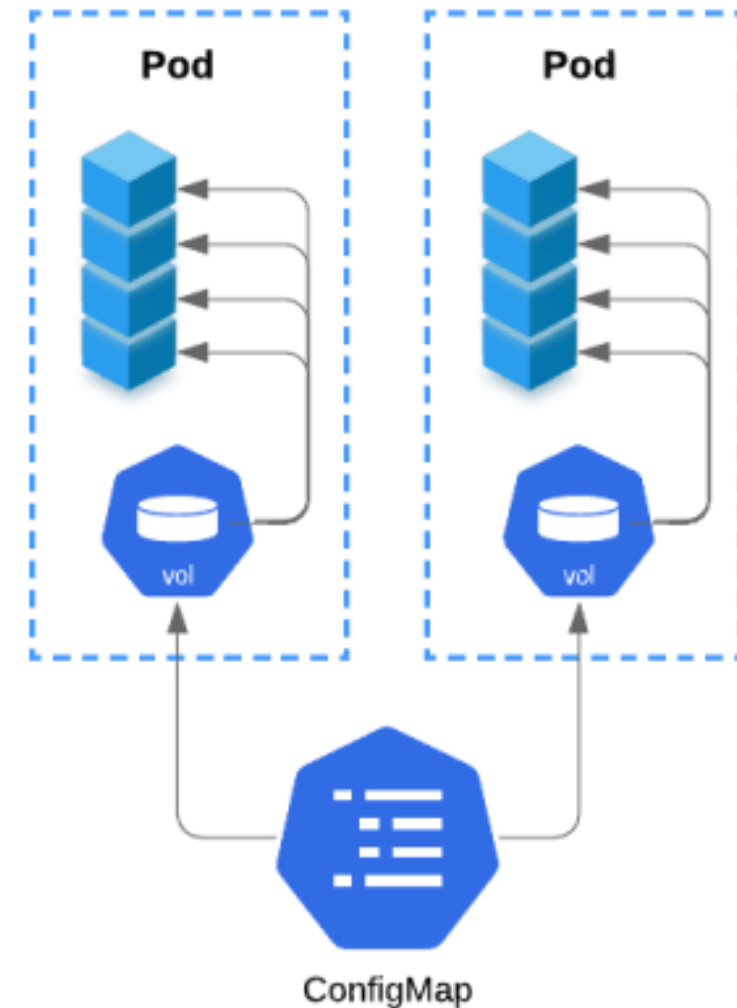
apiVersion: v1
kind: Service
metadata:
  name: demo
  labels:
    app: demo
spec:
  ports:
    - port: 8888
      protocol: TCP
      targetPort: 8888
  selector:
    app: demo
  type: ClusterIP
  
```

Injected Configurations → ConfigMaps

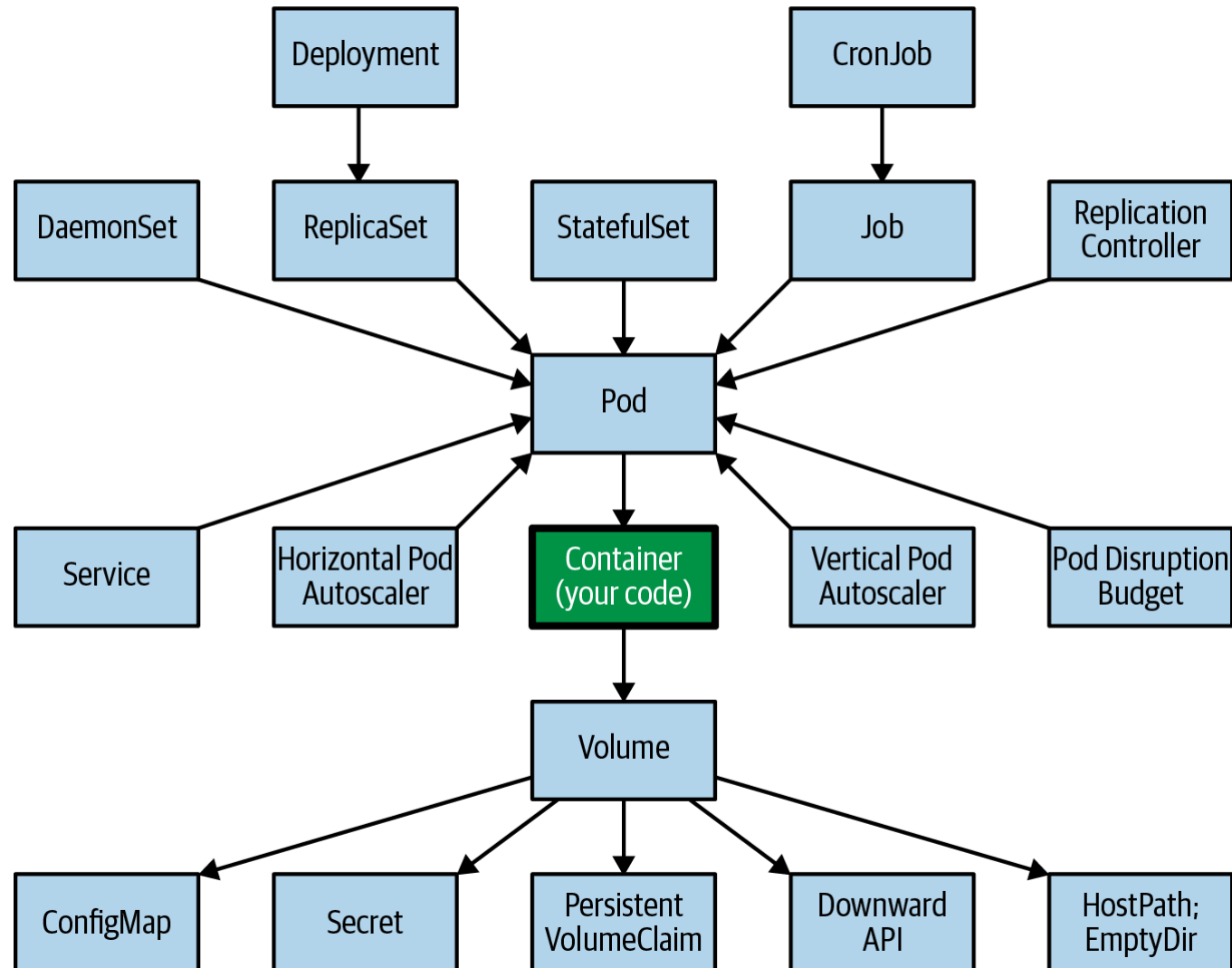
Configurations should be decoupled from running code → See 12Factor app

- ConfigMaps are static configuration files (json/yaml/properties/text) mounted to a path within a pod
- Can be provisioned as kubernetes resource
- Injecting is part of the framework

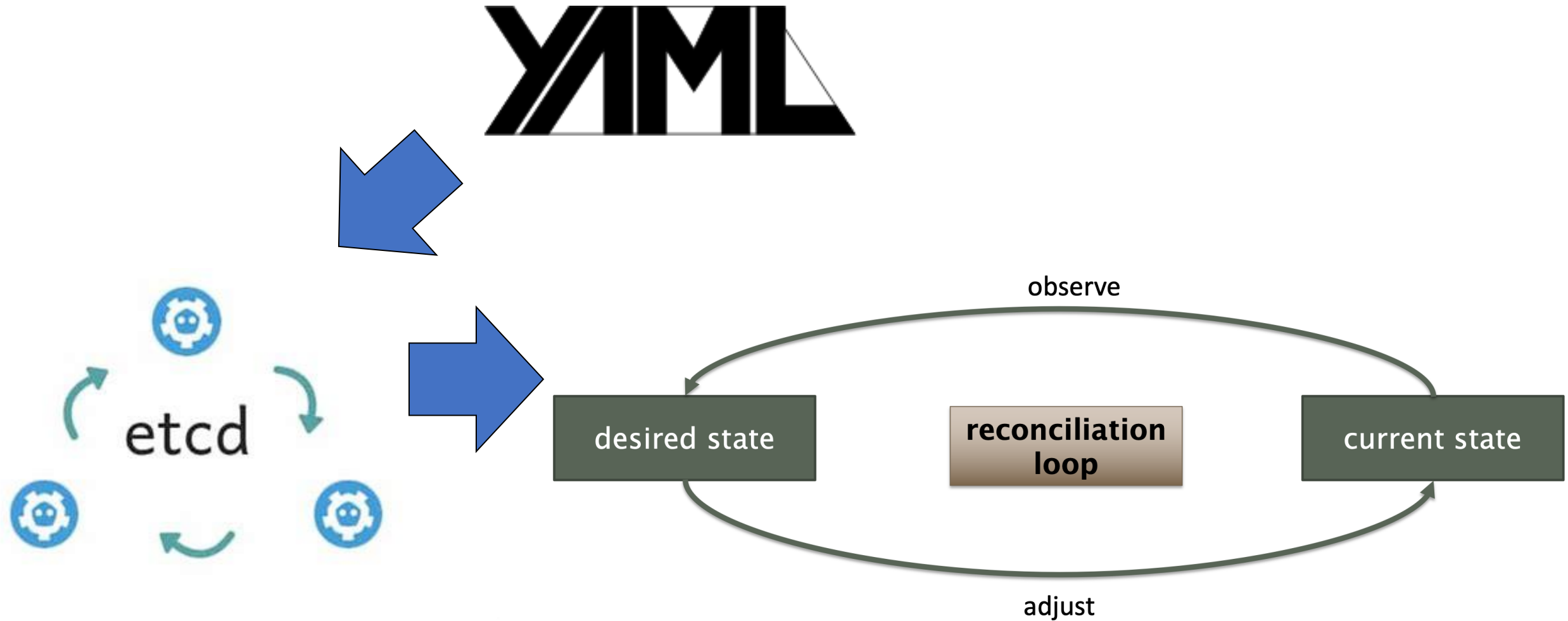
```
apiVersion: v1
kind: ConfigMap
metadata:
  name: mio-assortment-configmap
data:
  application-config.json:
    '{
      "spring.zipkin.baseUrl": "http://zipkin:9411/"
    }'
```



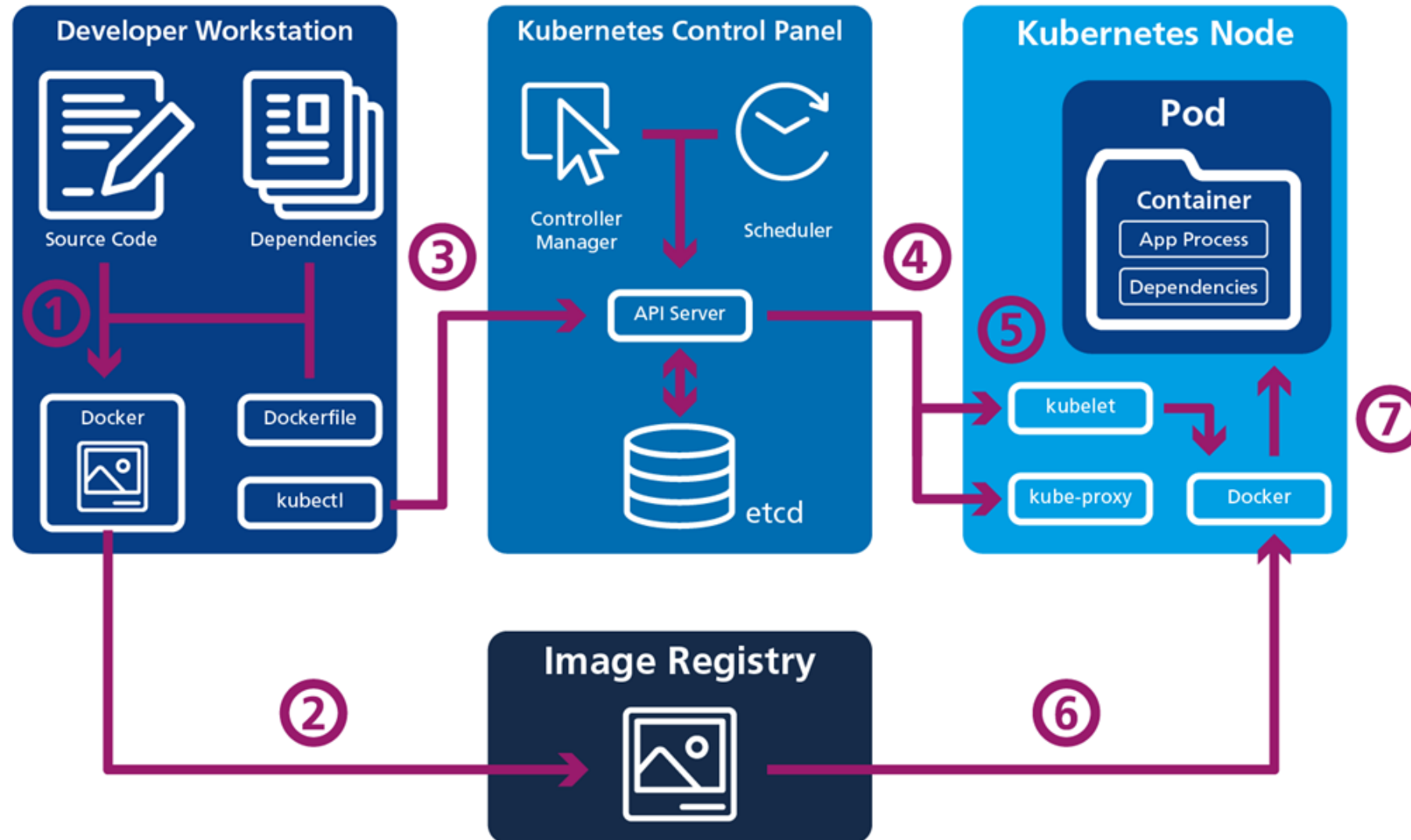
Defining any kind of resources



How elements are rolled out?



Workflow of Deployment



Common Commands

- `$ kubectl [command] [TYPE] [NAME] [flags]`
- `$ kubectl get pods`
- `$ kubectl describe pods`
- `$ kubectl edit pod my-pod`
- `$ kubectl apply -f my-pod.yaml`
- `$ kubectl logs -f my-pod`
- `$ kubectl exec my-pod -it /bin/sh`



<https://kubernetes.io/docs/tasks/tools/install-kubectl>

<https://kubernetes.io/docs/reference/kubectl/overview>

<https://github.com/cloudnativedevops/demo>

